

Deep Learning

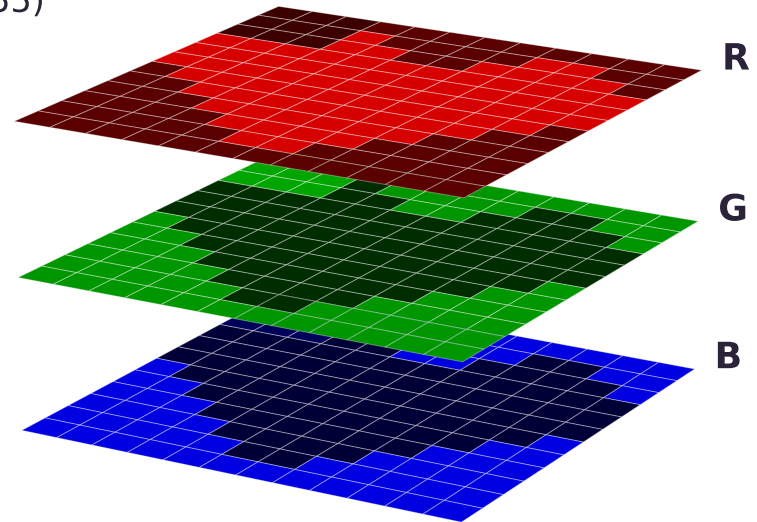
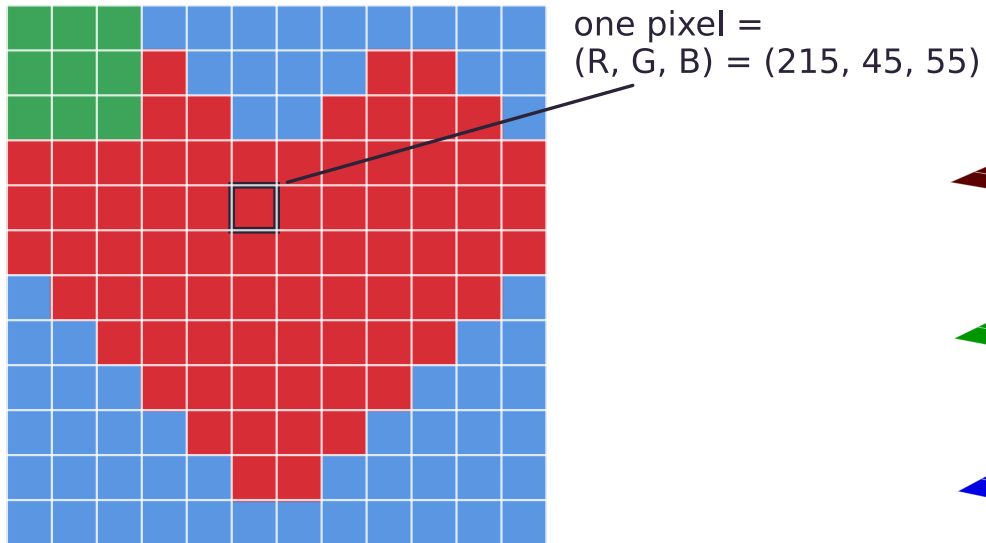
5. Images and Convolutional neural networks

Romain Tavenard

Preamble: What's an image?

An image = a grid of pixels

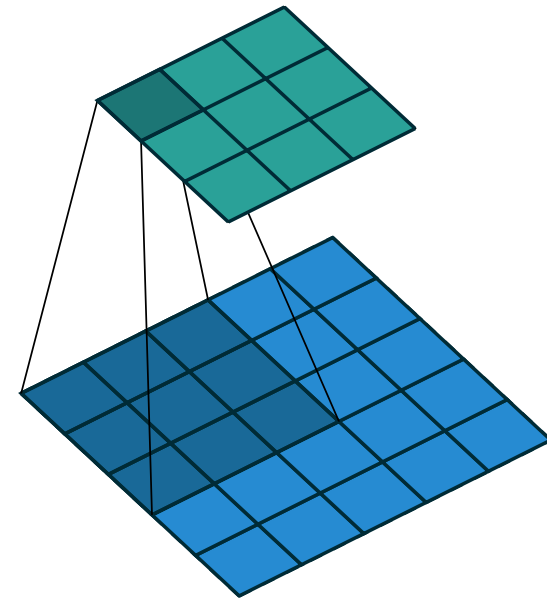
A pixel = 3 luminance values (R, G, B)



height × width × 3 channels

The convolution operator

- 2D convolution
 - Blue: input image
 - Gray: convolution kernel
 - Cyan: activation map
- Convolution operation = Dot product between
 - convolution kernel (aka filter)
 - subpart of the input



3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

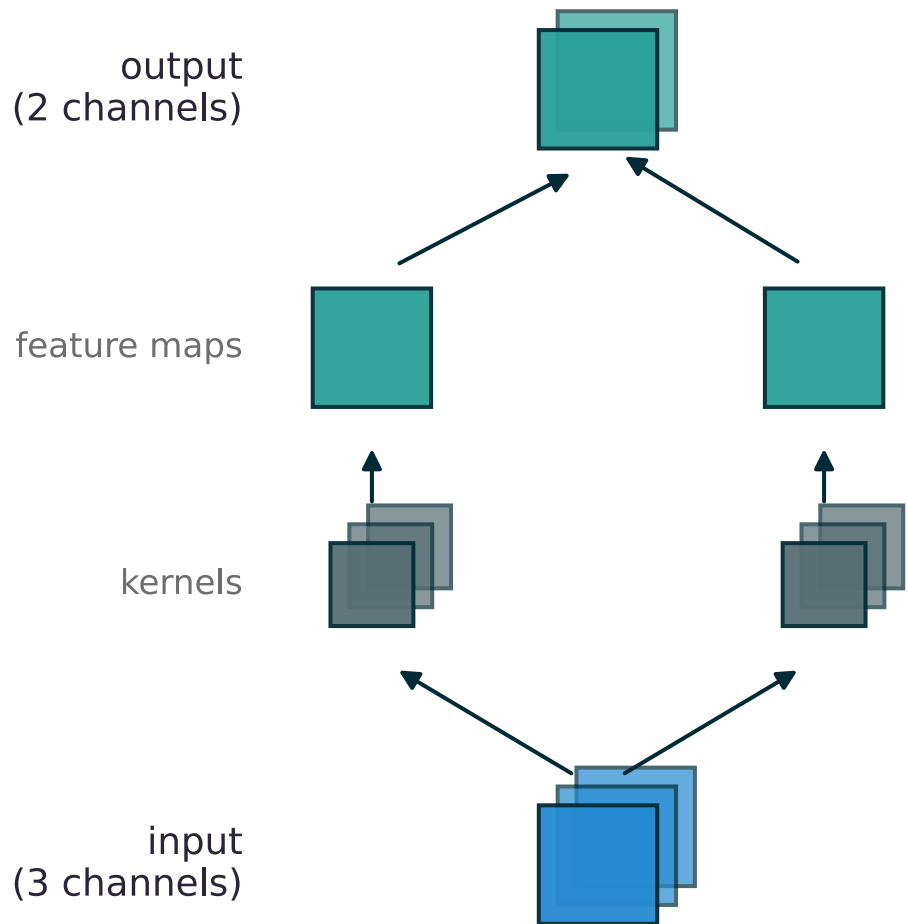
12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Convolutional layers in NN (1/2)

- A convolution layer is made of
 - convolution kernels
 - biases (1 per kernel)
 - an activation function
- Useful because
 - reduces #parameters (**parameter sharing**: same kernel reused everywhere)
 - e.g. dense neuron on 224×224 : ≈ 50000 weights vs 3×3 kernel: 9
 - encodes **translation equivariance** (translation in the input induces translation in the output)

Convolutional layers in NN (2/2)

- Multiple input channel case
 - sum the response over all channels
- Multiple kernel case
 - each kernel leads to one output channel



Convolutional layers in NN: hyper parameters

- Padding

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0 ₀	1 ₁	3 ₂	1	0
0	3	1 ₂	2 ₂	2 ₀	3	0
0	2	0 ₀	0 ₁	2 ₂	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	14.0	17.0	11.0	3.0
14.0	12.0	12.0	17.0	11.0
8.0	10.0	17.0	19.0	13.0
11.0	9.0	6.0	14.0	12.0
6.0	4.0	4.0	6.0	4.0

padding="same"

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

padding="valid"

- Strides

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

stride=1

3	3	2 ₀	1 ₁	0 ₂
0	0	1 ₂	3 ₂	1 ₀
3	1	2 ₀	2 ₁	3 ₂
2	0	0	2	2
2	0	0	0	1

12.0	17.0
9.0	14.0

stride=2

Computing the size of an activation map

- Assumption: no padding (“valid”), unit strides

$$W_{\text{out}} = W_{\text{in}} - W_k + 1$$

$$H_{\text{out}} = H_{\text{in}} - H_k + 1$$

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Output shrinks by kernel size – 1

Example:

- Input: 28×28 (MNIST digit)
- Kernel: 5×5
- Output: $(28 - 5 + 1) \times (28 - 5 + 1) = 24 \times 24$

With padding=“same”:

output stays 28×28

Pooling (aka subsampling) layers in NN

- **Why pool?**

- shrink spatial size → fewer computations
- robustness to small shifts (conv = equivariant, pooling → invariant)

- Max pooling / Average pooling

- Hyper-parameter
 - pool size

3	3	2	1	0	0
0	1	3	1	3	1
2	2	3	2	0	0
2	2	2	0	0	0
1	0	1	3	2	2
3	2	0	3	0	1

3.0	3.0	3.0
2.0	3.0	0.0
3.0	3.0	2.0

Max pooling example

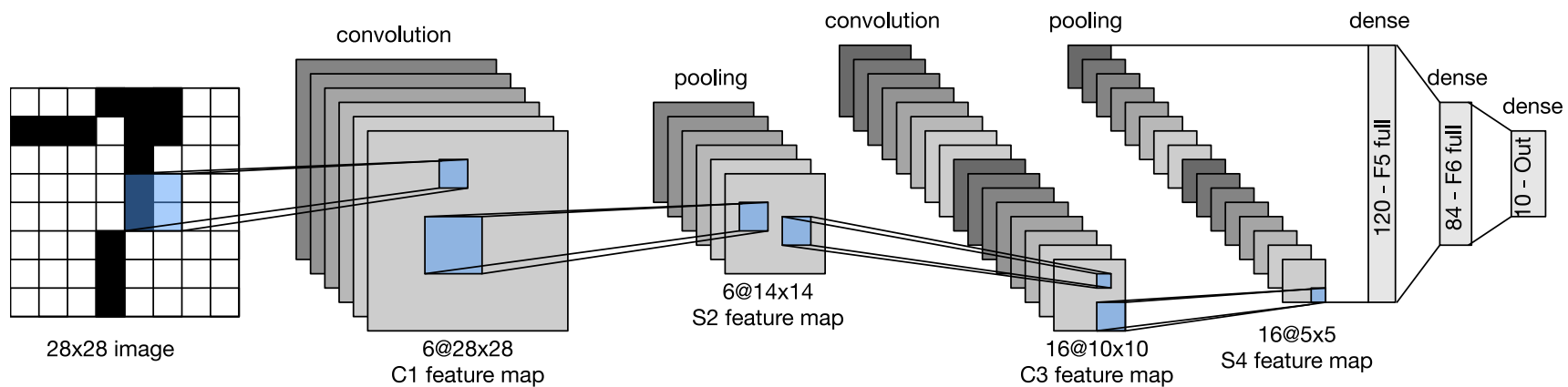
LeNet (1989)



MNIST dataset

10 classes

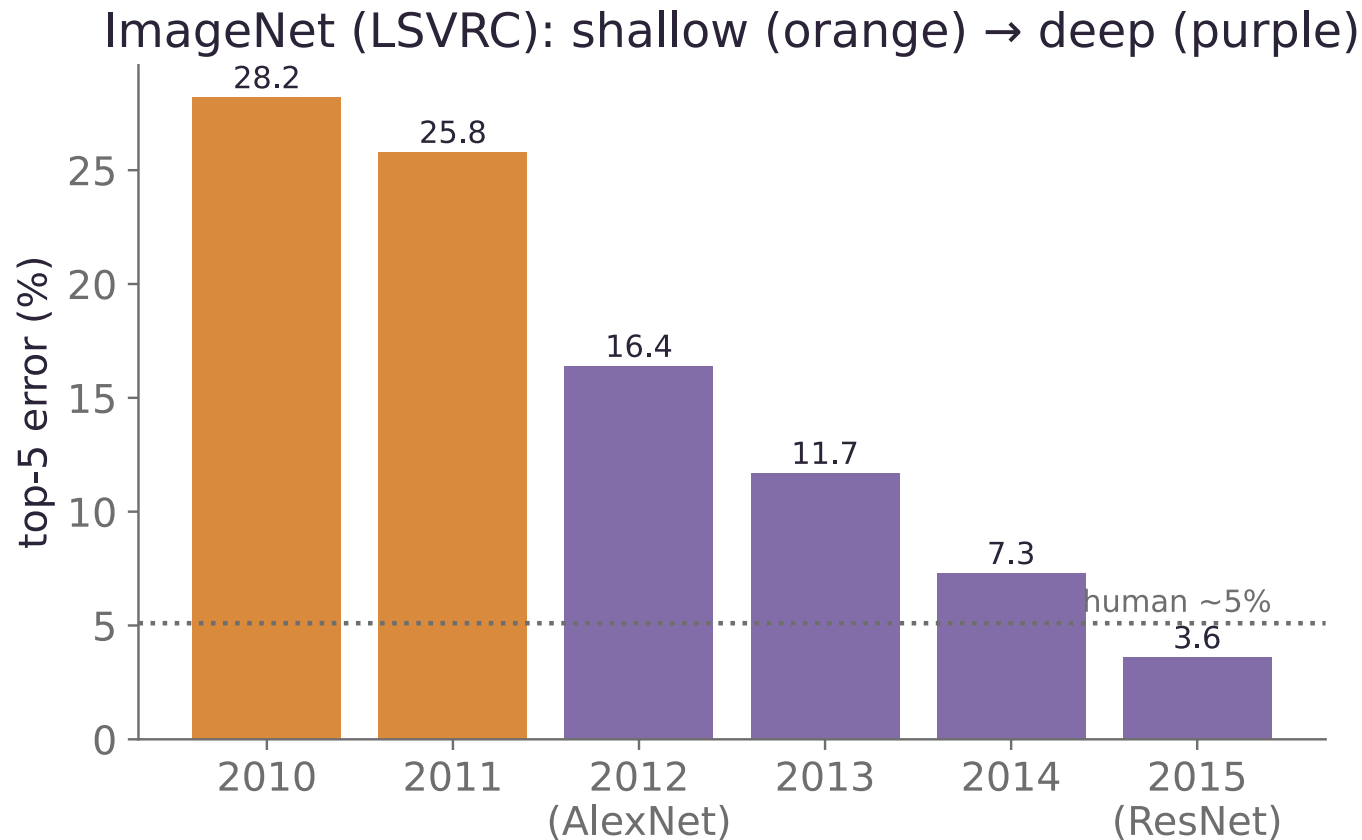
60,000 images



LeNet-5 architecture (LeCun et al., 1989). Source: Wikipedia

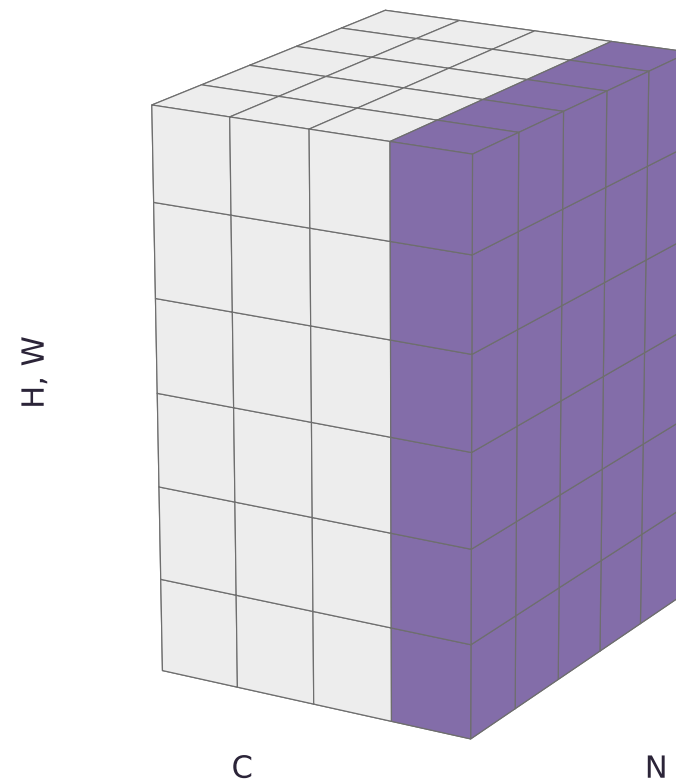
A drastic improvement on performance (ImageNet)

- ImageNet
 - 15M images
 - 22k classes
- LSVRC: Subset of ImageNet (1.2M images, 1k classes)



Batch Normalization for images

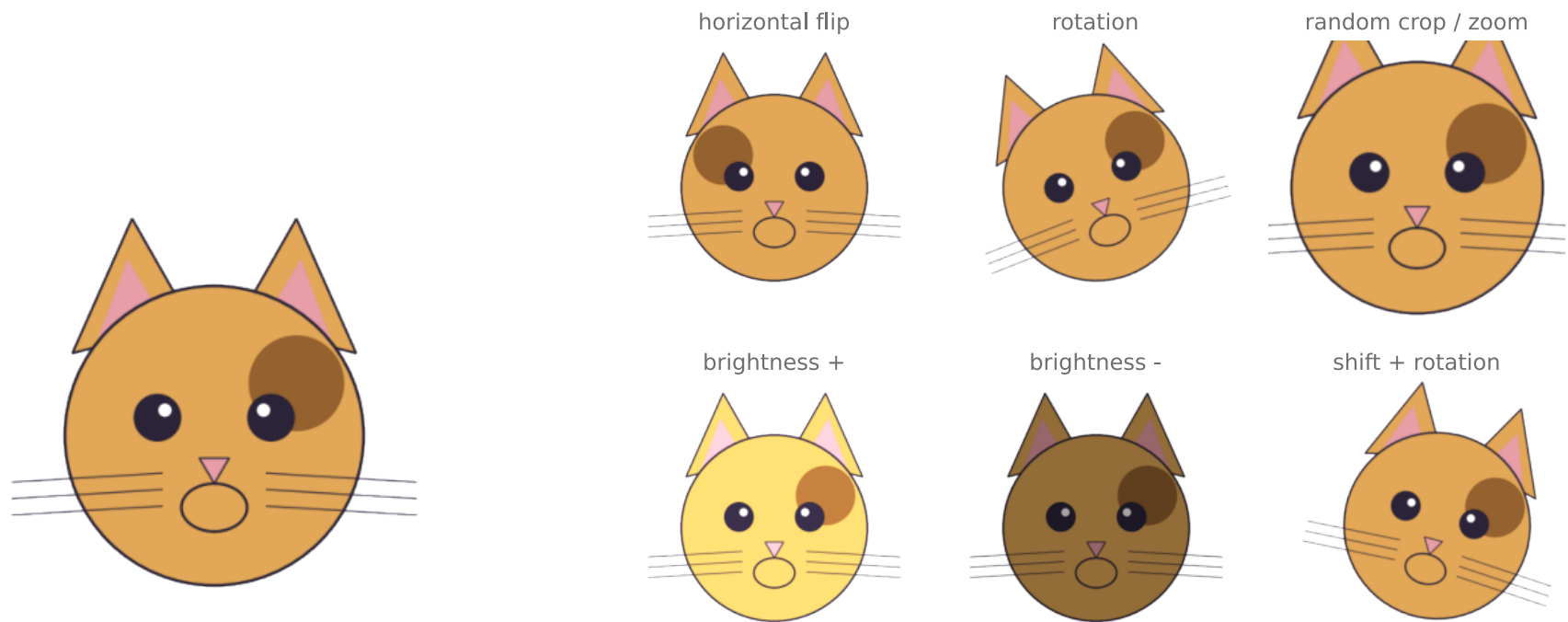
- Recall: BatchNorm normalizes activations (zero mean, unit variance), then rescales with learnable γ, β
- For a batch of images (tensor $N \times C \times H \times W$)
 - ▶ one mean/variance pair **per channel C**
- Effect: stabilizes/speeds up training; also acts as a (mild) regularizer



Batch Norm: for each channel (purple slice), statistics are computed over the batch and spatial dimensions (N, H, W)

Regularization: Data Augmentation

- Recall : big nets over-fit $\rightarrow$ regularize (early stopping, Dropout, BatchNorm)
- Images: one extra, very effective trick: **Data Augmentation**
- Principle: generate virtual training examples



Original image (label: “cat”)

6 augmented versions (same label)